

Operating Systems Background, Access Control

CS 465 · Introduction to Security and Privacy · Fall 2026

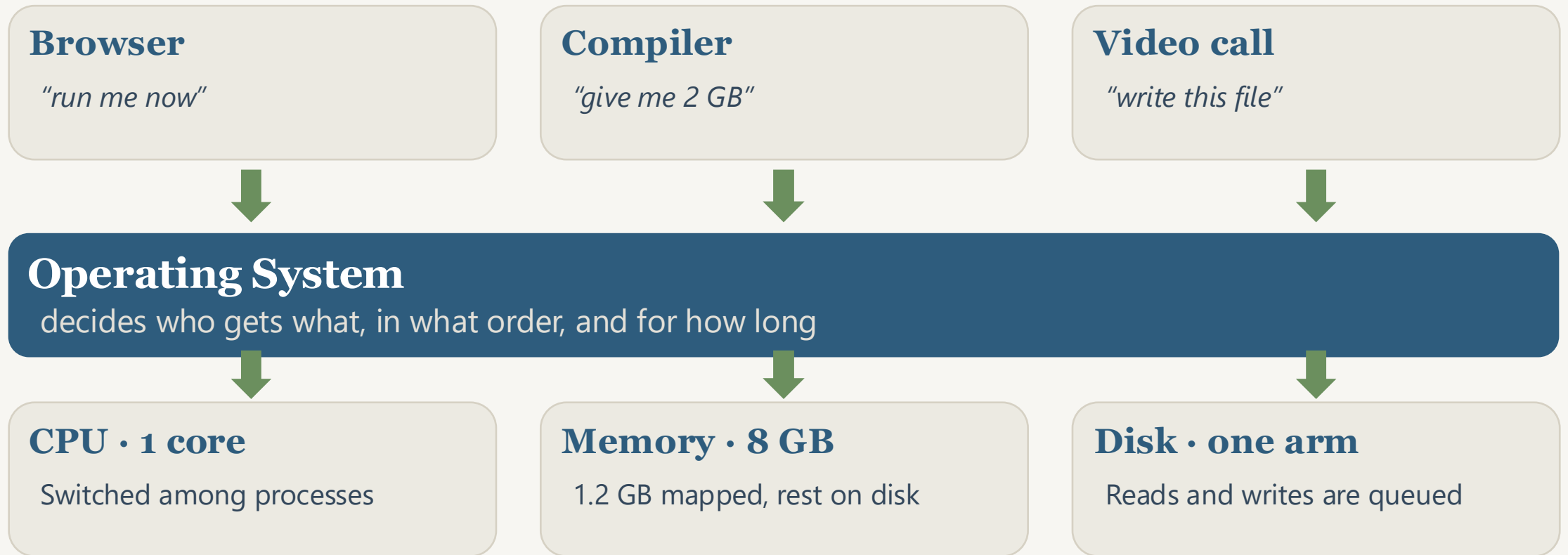
BYU



What Is an Operating System?

- The software layer between programs and hardware
- It takes a physical resource — CPU, memory, disk — and turns it into an easier-to-use virtual form, which is why the OS is sometimes called a virtual machine
- It is the **resource manager**: many programs want the CPU, memory, and devices at once, and the OS decides who gets what, and when
- Because it is the referee, it is also the enforcer: nothing but the OS stands between one program and another program's data

The OS as Resource Manager



The arbiter is also the referee.

The Three Easy Pieces

Virtualization

One CPU is made to look like many; each process gets its own memory

Concurrency

Many things happen at once — and still have to be correct

Persistence

Data survives crashes, reboots, and power loss

Security cuts across all three. Each abstraction is also a boundary someone will try to cross.

The User / Kernel Boundary

User mode — applications

Your program has no direct access to hardware; it calls the standard library, which issues system calls.



trap into the kernel

return to user mode



Kernel mode — the operating system

Privileged: touches devices, page tables, and other processes' memory. Every request is checked before it is granted.

The system call is the only door into the kernel — which is why attacks aim straight at it.

OS Design Goals

- Abstraction: build virtual forms of the CPU, memory, and disk that are pleasant to use
- Performance: add as little overhead as possible while doing it
- Protection and isolation: one process must not be able to harm another, or the OS itself
- Reliability: the OS has to run for months without failing
- And also: energy efficiency, security against malicious applications, and mobility

Why Security Lives in the OS

Isolation

Processes cannot read or write each other's memory — the OS enforces the wall.

Mediation

Every access to a file, device, or network socket goes through the kernel.

Identity

The OS authenticates users and enforces access control on their behalf.

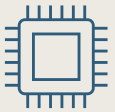
Trusted base. If the kernel is compromised, nothing running above it can be trusted.

OS background

Three sections from Operating Systems: Three Easy Pieces (OSTEP)

Processes

A running program: state, memory, and I/O



CPU Scheduling

Which process runs, and for how long



Address Spaces

How memory is virtualized and protected



Processes

The illusion of unlimited CPUs



What Is a Process?

- A process is a running program
- Processes let the OS pretend the machine has an unlimited number of CPUs, so many programs appear to run simultaneously
- Each process has state (registers, stack), memory (its address space), and I/O
- A process has both a stack and a heap
- The OS exposes an API to create, destroy, wait for, and control processes

Process States



The OS keeps track of every process in a data structure, recording its current state, saved registers, and memory.

CPU Scheduling

Which process runs, and for how long



Measuring a Scheduler

Turnaround time

- time completed – time arrived
- Measures how long a job waits to finish
- Optimized by running short jobs first (SJF, STCF)

Response time

- time first run – time arrived
- Measures how quickly a job gets attention
- Optimized by rotating quickly among jobs (Round Robin)

Scheduling algorithm: the part of the OS that decides which process runs and for how long.

Compare the book's turnaround-time diagrams for each algorithm.

Scheduling Algorithms

Algorithm	How it picks	Trade-off
FIFO	First in, first out — arrival order	One long job at the front inflates everyone's turnaround time
SJF	Shortest job first	Needs to know job lengths; short jobs arriving late still wait
STCF	Shortest time-to-completion first; preempts when a shorter job arrives	Strong turnaround time, weak response time
Round Robin	Small time slice to each job in turn	Much better response time, worse turnaround time

Modern schedulers are far more complex — see the multi-level feedback queue in the next chapter.

Address Spaces

Virtualizing memory



How Memory Use Evolved

1

Early systems

One process at a time used all the physical memory the OS was not using.

2

Multiprogramming

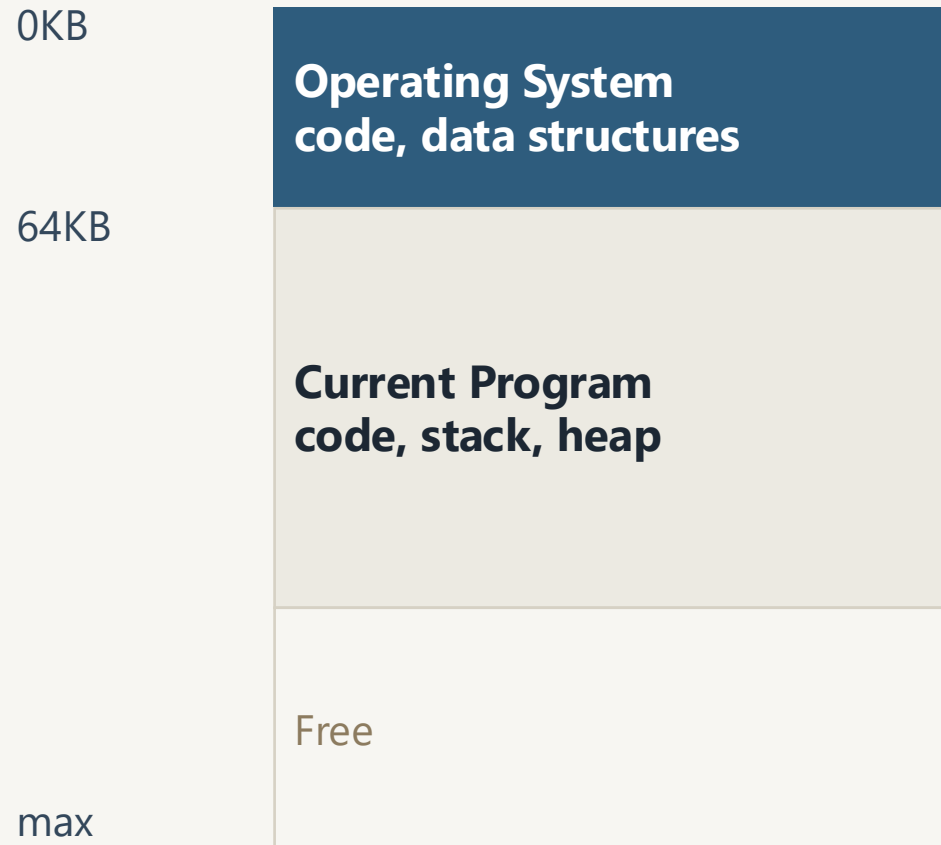
Several processes sit in memory at once, so the CPU can switch among them.

3

Virtual address spaces

Every process gets its own address space; the OS maps virtual addresses to physical memory.

One Program at a Time



Swapping in a new program is very expensive – disks are slow.

Multiprogramming

Leave processes resident

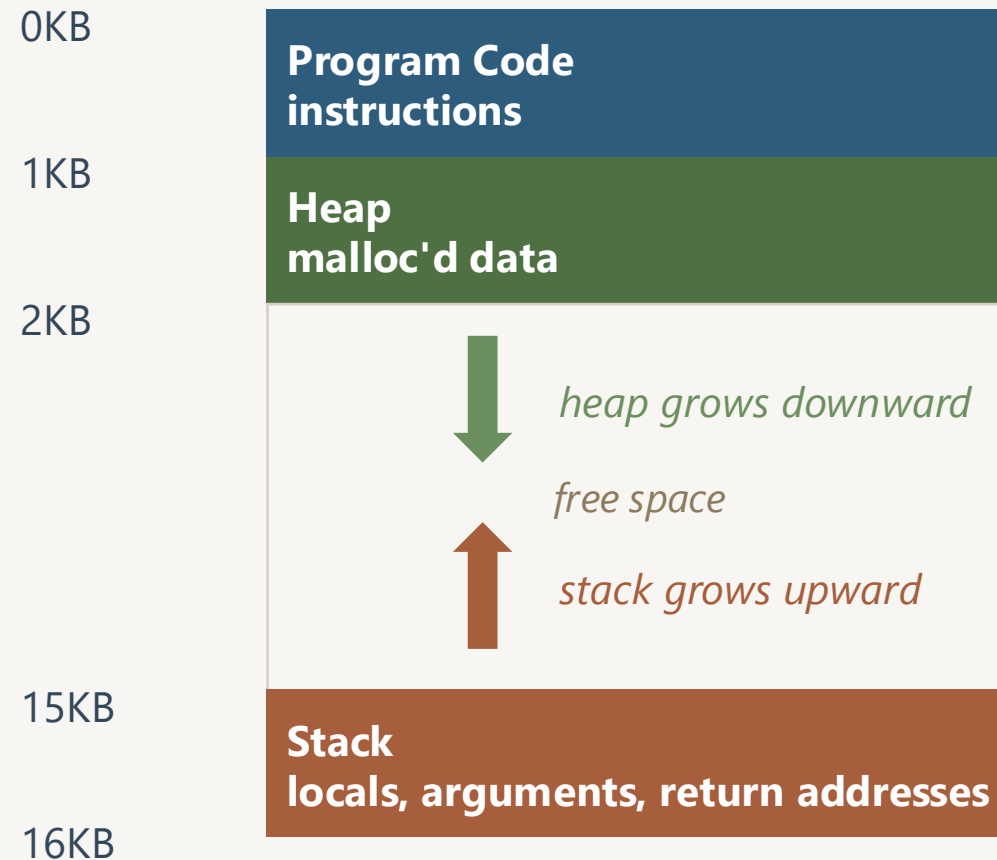
fast — and now they are neighbors

Operating System
Process A
Process B
Process C

Need memory protection to protect process A from processes B and C.

The more processes, the fewer resources for each.

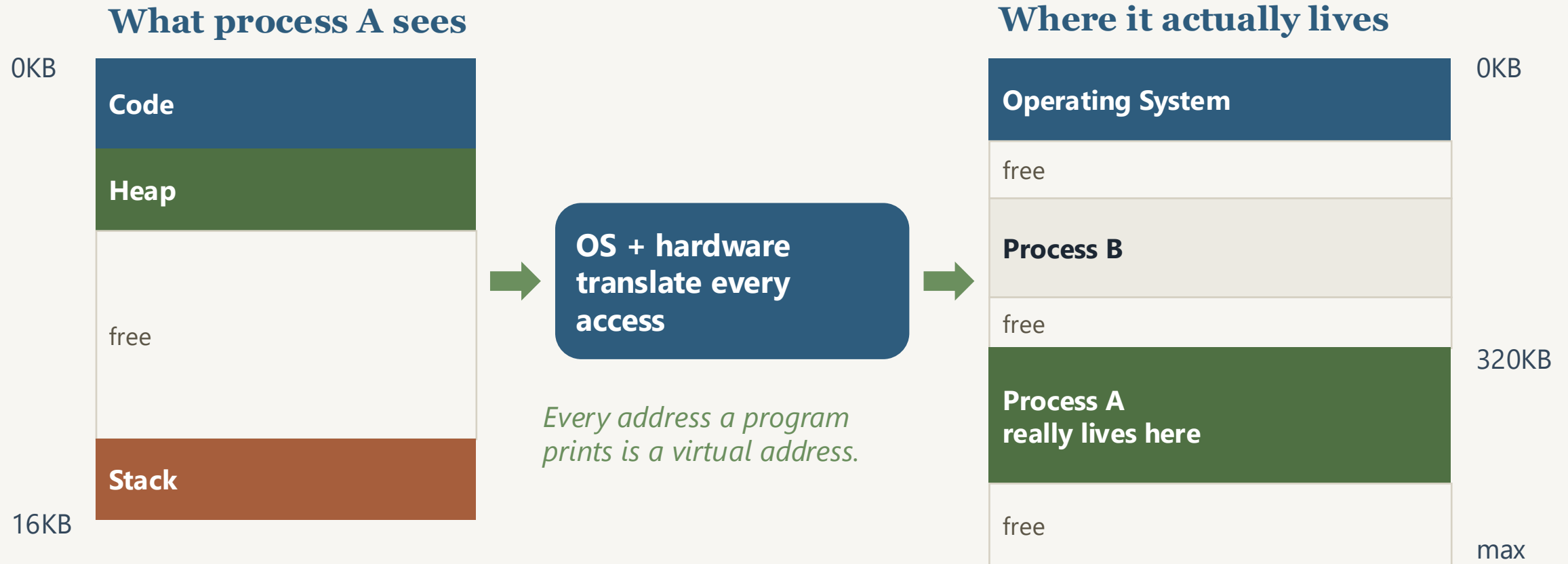
Virtual Address Space



Every process believes it starts at address 0.

Heap and stack are the two regions that grow — so they are placed at opposite ends and grow toward each other, with the free space between them.

Virtual Is Not Physical



Three Goals of Virtual Memory

Transparency

The program should never notice. It writes to address 0x1000 and it works — no hint that the OS moved it, split it, or paged it out.

Efficiency

Virtualization must cost little in time or space, so the OS leans on hardware support such as the TLB and page tables.

Protection

One process cannot read or write another's memory, or the OS's. Each address space is isolated — a private world.

Protection is the security goal

Isolation is not a bonus feature of virtual memory — it is the reason we can run untrusted code at all.

Every Address You See Is Virtual

```
int main(int argc, char *argv[]) {  
    printf("location of code : %p\n", main);  
    printf("location of heap : %p\n", malloc(1));  
    int x = 3;  
    printf("location of stack: %p\n", &x);  
    return x;  
}
```

what it prints

```
location of code : 0x40057d  
location of heap : 0xcf2010  
location of stack: 0x7fff9ca45fc4
```

None of these addresses are real

Code low, heap just above it, stack far away at the top — exactly the address-space picture. All three are virtual addresses, translated before any physical byte is touched.

What the OS Does With Memory

- Translation: maps each process's virtual addresses onto physical memory
- Swapping: moves memory in and out of physical space based on what the running process needs
- Protection: keeps a process from reading or writing another process's memory
- Remember: the OS is a resource manager

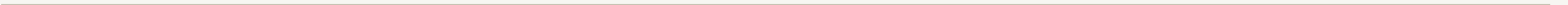
**Want to learn more
about operating
systems?**

Take CS 345



OS Security Goals and Principles

What are we trying to accomplish and how do we go about it?



Why OS Security Is Hard

- Everything runs on top of the OS — if it is insecure, nothing above it can be trusted
- The more complex a program, the harder it is to secure, and the OS is enormous
- We assume the OS is secure — but a single flaw in a single program can hand the whole machine to an attacker

AN ASIDE

The browser has become an operating system, with the same complexity problem

What the OS Can Do

Examine or alter any process's memory

Read, write, or delete any file on any writable medium

Reschedule or halt execution of any process

Send any message anywhere — including an altered one

Enable or disable any peripheral device

Give one process access to another's resources

Take away any resource a process controls

Answer any system call with a maximally harmful lie

Every one of these is a legitimate OS power — security is critical to avoid abuse

Security Goals: C, I, A

Confidentiality

Nobody learns what they are not allowed to learn. Your files, your memory, your keys stay yours.

Broken when: one process reads another's memory.

Integrity

Nobody changes what they are not allowed to change — and if they try, you can tell.

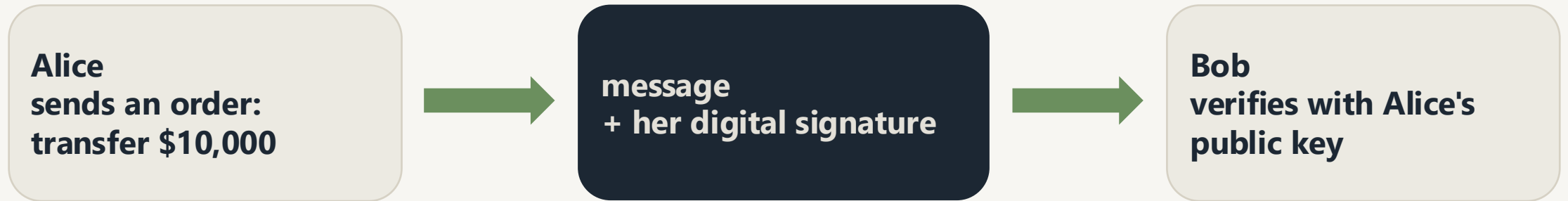
Broken when: malware quietly rewrites a system binary.

Availability

The system keeps working for the people who are allowed to use it.

Broken when: a runaway process starves everyone else.

Non-Repudiation



Later, the dispute

**"I never sent that order."
— Alice**

**Only Alice's private key could have produced
that signature.**

Integrity says the message was not altered. Non-repudiation says who sent it — provably, to someone who trusts neither party.

Saltzer and Schroeder's Design Principles

Economy of mechanism — as small and simple as possible

Fail-safe defaults — deny by default, grant explicitly

Complete mediation — check the policy on every access

Open design — assume the attacker knows the design

Separation of privilege — two parties or credentials, such as 2FA

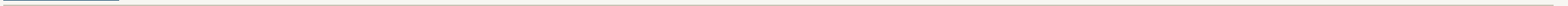
Least privilege — the lowest privilege that does the job

Least common mechanism — separate structures per user

Acceptability — if it hurts to use, people route around it

OS Access Control

Who is allowed to do what — and who decides



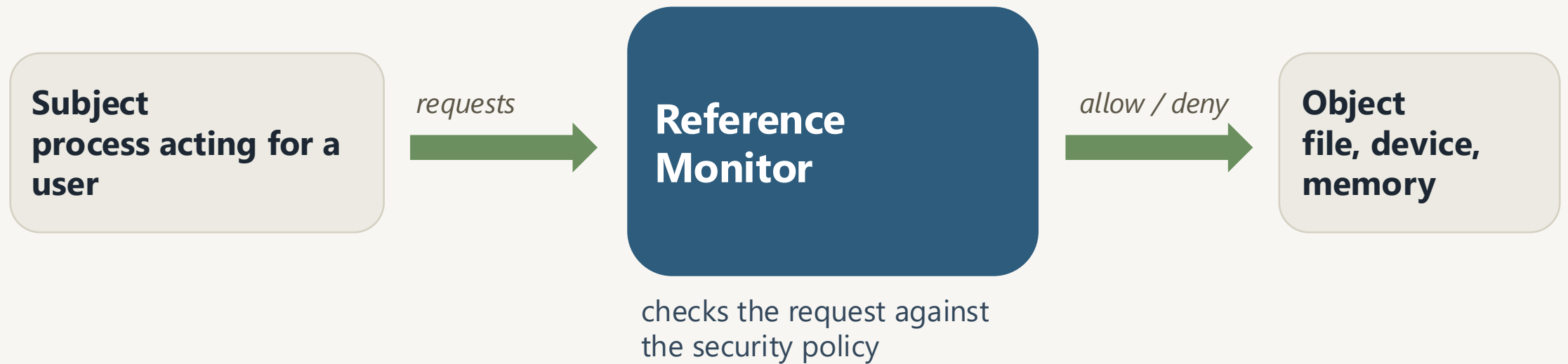
Access Control: The Basic Idea

- 1 Figure out whether the request fits the security policy
- 2 If it does, perform the operation. If it does not, make certain it never happens

THE QUESTION THE OS HAS TO ANSWER

A process asks to open a file for reading and writing. Should the OS let it — and how does it know?

The Reference Monitor



Complete mediation in practice

A process wants to open a file for reading and writing. Should the OS let it? Every such question goes through the monitor — if it can be bypassed even once, the policy is decoration.

Two Ways to Write It Down

Access control lists

For each object, list the subjects that may touch it.

Unix does this in nine bits per file — r w x for user, group, and other — kept in file metadata.

Hard question: what can Bilbo reach? You must search everywhere.

```
rwX  r-x  r--  owner  group
```

nine bits, the whole policy for one file

Capabilities

For each subject, list the objects it may touch — a ring of keys the process carries.

The OS must hold the keys, or a process could mint its own.

Hard question: who can reach this file? That one is expensive.

```
read /home/bilbo  write /tmp
```

the keys one process is carrying

Who Decides the Policy?

Mandatory

The organization sets access for every object. Users cannot loosen it, no matter who owns the file.

Example: a classified file stays classified even if its author wants to share it.

Discretionary

The owner of an object decides who else may use it — the Unix model.

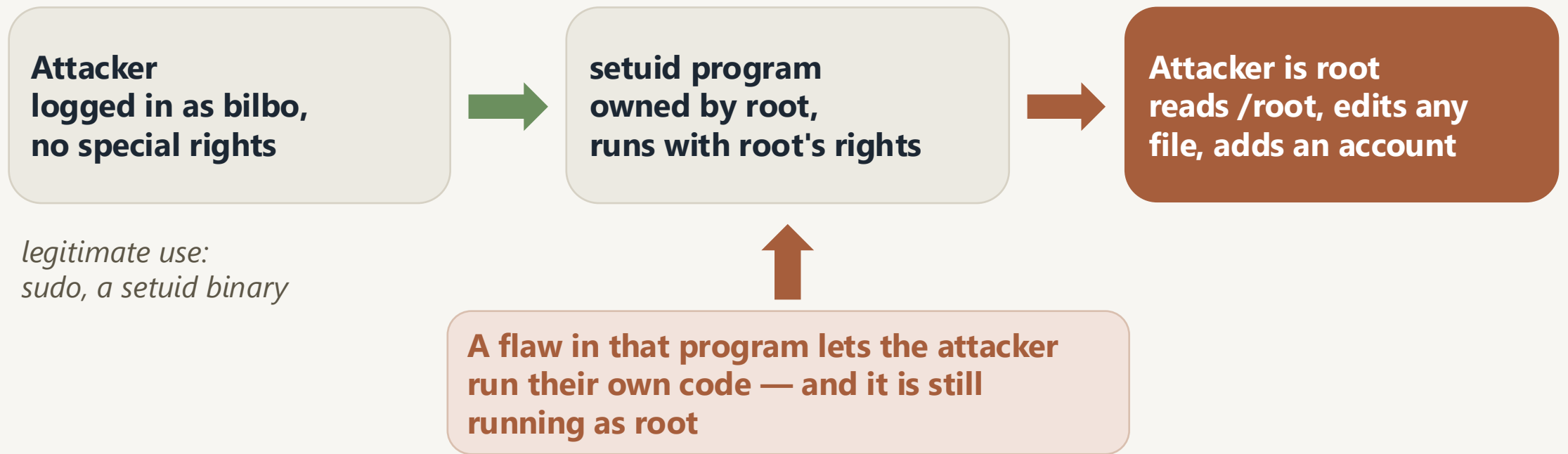
Example: `chmod 644`, and Bilbo alone decides who reads his file.

Role-based

Access follows the role, not the person. A customer orders, a manager restocks, an admin grants roles.

Example: move someone into the admin role and their access moves with them.

Privilege Escalation



Every escalation path is a hole in least privilege — which is exactly why attackers hunt for them